



Arquitectura De Computadoras

Fase 3

Proyecto Final

Profesor: Jorge Ernesto Lopez Arce Delgado

José Manuel Alvarez Flores

Héctor Ventura Pérez

Sección D03

Introducción

Características generales de procesadores MIPS

Los procesadores desarrollados por MIPS Technologies se basan en la arquitectura RISC (Reduced Instruction Set Computer), cuyo objetivo principal es simplificar las instrucciones para lograr una ejecución más rápida y eficiente. A diferencia de otras arquitecturas con instrucciones complejas, MIPS utiliza un conjunto reducido de instrucciones simples, permitiendo que la mayoría puedan ejecutarse en un solo ciclo de reloj. Esta filosofía facilita el diseño interno del procesador y mejora su rendimiento general.

Una de las características más importantes de los procesadores MIPS es que sus instrucciones tienen una longitud fija de 32 bits. Esto simplifica la etapa de decodificación y favorece la implementación de técnicas como el pipeline. Además, MIPS sigue una arquitectura tipo load/store, lo que significa que únicamente las instrucciones de carga y almacenamiento pueden acceder directamente a la memoria. Las operaciones aritméticas y lógicas se realizan exclusivamente entre registros internos del procesador, reduciendo el tiempo de acceso a memoria y aumentando la velocidad de ejecución.

El procesador cuenta normalmente con 32 registros de propósito general de 32 bits, utilizados para almacenar datos temporales, direcciones y resultados intermedios. Algunos registros tienen funciones específicas, como \$zero, cuyo valor siempre es cero, o \$ra, empleado para almacenar direcciones de retorno en llamadas a funciones. El uso intensivo de registros es una característica clave de la arquitectura MIPS, ya que minimiza accesos innecesarios a memoria.

Otra característica fundamental es el uso del pipeline, una técnica que divide la ejecución de instrucciones en varias etapas, como búsqueda de instrucción, decodificación, ejecución, acceso a memoria y escritura de resultados. Gracias a esto, múltiples instrucciones pueden procesarse simultáneamente en diferentes etapas, aumentando considerablemente el rendimiento del procesador.

MIPS también se distingue por utilizar modos de direccionamiento simples y pocos formatos de instrucción. Principalmente emplea tres formatos: tipo R para operaciones entre registros, tipo I para instrucciones inmediatas y acceso a memoria, y tipo J para instrucciones de salto. Esta organización hace que el diseño del hardware sea más sencillo y fácil de entender.

ISA

El set de instrucciones específico usado para este proyecto es el siguiente:

Instrucción	Tipo	Sintaxis
Lw	I	lw \$rt, offset (base)
Sw	I	sw \$rt, offset (base)
slt	R	slt \$rd, \$rs, \$rt
Slti	I	slti \$rt, \$rs, immediate
beq	I	beq \$rs, \$rt, offset
bne	I	bne \$rs, \$rt, offset
j	J	j target
nop	R	nop
bgtz	I	bgtz \$rs, offset

Instrucción	Tipo	Sintaxis
Add	R	add \$rd, \$rs, \$rt
Sub	R	sub \$rd, \$rs, \$rt
Or	R	or \$rd, \$rs, \$rt
And	R	and \$rd, \$rs, \$rt
Addi	I	addi \$rt, \$rs, immediate
Subi	I	addi \$rt, \$rs, -immediate
Ori	I	ori \$rt, \$rs, immediate
Andi	I	andi \$rt, \$rs, immediate

Algoritmo en ensamblador

El algoritmo elegido para su implementación en ensamblador y posterior traducción a instrucciones de nuestra arquitectura fue la multiplicación matricial, debido a que en nuestro set de instrucciones no hay multi, se harán las multiplicaciones con sumas acumuladas.

La multiplicación matricial es una operación fundamental del álgebra lineal utilizada en áreas como programación, gráficos por computadora, inteligencia artificial, procesamiento digital y arquitectura de computadoras. El objetivo del algoritmo de multiplicación matricial es obtener una nueva matriz a partir de la combinación de dos matrices compatibles entre sí.

Para que dos matrices puedan multiplicarse, el número de columnas de la primera matriz debe ser igual al número de filas de la segunda. Si una matriz A tiene dimensiones $m \times n$ y una matriz B tiene dimensiones $n \times p$, entonces el resultado será una matriz C de dimensiones $m \times p$, en lo que a nuestra implementación respecta, se trabajará con una matriz de 3×3 .

El procedimiento consiste en calcular cada elemento de la matriz resultante mediante el producto punto entre una fila de la primera matriz y una columna de la segunda. Matemáticamente, cada elemento de la matriz resultado se obtiene con la siguiente expresión:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

Pseudocódigo

Para i ← 0 hasta 2 hacer

 Para j ← 0 hasta 2 hacer

 C[i][j] ← 0

 Para k ← 0 hasta 2 hacer

 C[i][j] ← C[i][j] + (A[i][k] * B[k][j])

 Fin Para

 Fin Para

Fin Para

Con sumas acumuladas sería:

Para $i \leftarrow 0$ hasta 2 hacer

Para $j \leftarrow 0$ hasta 2 hacer

suma $\leftarrow 0$

Para $k \leftarrow 0$ hasta 2 hacer

producto $\leftarrow 0$

Para $l \leftarrow 1$ hasta $B[k][j]$ hacer

producto $\leftarrow$ producto + $A[i][k]$

Fin Para

suma $\leftarrow$ suma + producto

Fin Para

$C[i][j] \leftarrow$ suma

Fin Para

Fin Para

Objetivo

Diseñar un “datapath” con arquitectura tipo MIPS de 32 bits capaz de ejecutar las 28 instrucciones previamente definidas por nosotros.

Objetivos particulares

- Fase 1:
 - Diseñar los módulos necesarios para crear el datapath que permita ejecutar las instrucciones de tipo R de la tabla 1 y tabla 2.
- Fase 2:
 - Agregar los módulos necesarios al datapath para poder ejecutar las instrucciones de tipo I de la tabla 1 y tabla 2.
- Fase 3:
 - Agregar los módulos necesarios para completar el datapath de MIPS de 32 bits para que sea capaz de ejecutar las instrucciones tipo J.
- Fase 4:
 - Crea un algoritmo en código ensamblador, utilizando solo las instrucciones de la tabla 1 y tabla 2.
 - Dicho código debe ser “decodificado” a código binario/máquina, el cual será pre-cargado a la memoria de instrucciones para que el datapath lo ejecute.
 - Crear un programa/script que realice la “decodificación” de su código ensamblador (se recomienda Python).

Desarrollo

En este proyecto se desarrolló un datapath segmentado de 32 bits basado en la arquitectura MIPS, implementado en Verilog y organizado en cinco etapas principales: IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access) y WB (Write Back). Cada etapa se conectó mediante registros intermedios o buffers de pipeline (IF_ID, ID_EX, EX_MEM y MEM_WB) sincronizados con el reloj principal del sistema, permitiendo que varias instrucciones se ejecutaran de manera simultánea en diferentes fases del pipeline.

Inicialmente se diseñaron los módulos básicos necesarios para el funcionamiento del procesador, como el contador de programa (PC), la memoria de instrucciones, el banco de registros, la ALU, las memorias de datos y las unidades multiplexoras. Posteriormente se implementó la unidad de control (UCtrl) encargada de generar las señales necesarias para cada tipo de instrucción soportada por el datapath.

El procesador fue diseñado para ejecutar instrucciones tipo R y tipo I del conjunto MIPS, incluyendo add, sub, and, or, slt, addi, andi, ori, slti, lw, sw, beq, j y nop. Para ello se implementó una unidad de control de la ALU (ALUCtrl) que permite seleccionar la operación adecuada dependiendo del opcode y del campo func de las instrucciones tipo R.

Durante el desarrollo también se implementó el soporte para instrucciones de salto y bifurcación. En el caso de beq, se agregó el cálculo de la dirección de branch utilizando el desplazamiento del inmediato y la suma con el valor de PC+4. Para la instrucción j, se añadió un multiplexor adicional para seleccionar la nueva dirección del contador de programa utilizando el formato de salto de MIPS. Además, se implementó una señal de flush en el registro IF_ID para limpiar instrucciones incorrectas dentro del pipeline cuando ocurre un branch o jump.

Otro aspecto importante fue el manejo de los inmediatos. Algunas instrucciones como addi, slti, lw, sw y beq utilizan extensión de signo, mientras que instrucciones lógicas como andi y ori requieren extensión con ceros. Por ello se implementaron los módulos SignExtend y ZeroExtend, junto con un multiplexor controlado mediante la señal ExtOp.

Para probar el funcionamiento del procesador se desarrolló un programa ensamblador capaz de realizar una multiplicación matricial de matrices de

3x3. Debido a que el datapath no implementa una instrucción de multiplicación, el producto entre elementos se realizó mediante sumas repetidas utilizando ciclos anidados. El programa utiliza instrucciones de carga, almacenamiento, operaciones aritméticas, saltos y bifurcaciones, permitiendo comprobar el funcionamiento correcto de todas las etapas del pipeline.

Adicionalmente se desarrolló un ensamblador en Python encargado de traducir el código ensamblador MIPS a instrucciones binarias de 32 bits compatibles con la memoria de instrucciones del datapath. Este programa reconoce labels, calcula offsets para instrucciones beq, genera direcciones para instrucciones j y convierte automáticamente cada instrucción a su representación binaria correspondiente. El resultado se almacena en el archivo instrucciones.txt, el cual es cargado por la memoria de instrucciones mediante \$readmemb.

Finalmente, se realizaron simulaciones en ModelSim para verificar el comportamiento del pipeline, el flujo de instrucciones y los resultados obtenidos en memoria y registros. También se comprobó el funcionamiento de las instrucciones soportadas y la correcta ejecución del programa de multiplicación matricial.

Conclusiones

José Manuel

Durante este proyecto se logró comprender el funcionamiento interno de un datapath con arquitectura MIPS y la manera en que se ejecutan las instrucciones. Se implementaron instrucciones tipo R, tipo I y tipo J, además de la memoria de instrucciones, banco de registros, ALU, buffers, etc. También se aprendió la importancia del control de flujo dentro de un pipeline.

Además se desarrolló un programa ensamblador y un decodificador en Python para transformarlo en binario de 32 bits para que este pueda ser ejecutado en ModelSim. Gracias a esto se pudo ejecutar un programa de multiplicación matricial, verificando el correcto funcionamiento del datapath.

Héctor

El desarrollo de este proyecto permitió comprender de manera práctica el funcionamiento de un procesador MIPS de 32 bits basado en arquitectura RISC, así como la importancia de cada etapa del pipeline en la ejecución eficiente de instrucciones. Mediante la implementación del datapath en Verilog se integraron módulos como la ALU, el banco de registros, las memorias y la unidad de control para soportar instrucciones tipo R, I y J.

Además, el algoritmo de multiplicación matricial permitió comprobar el funcionamiento del procesador utilizando operaciones aritméticas, acceso a memoria, saltos y bifurcaciones. Finalmente, las simulaciones realizadas validaron el correcto comportamiento del pipeline y de las instrucciones implementadas, reforzando conocimientos sobre arquitectura de computadoras, lenguaje ensamblador y diseño digital.

Referencias

Gasparetto, A. (2014). MIPS instruction set. Università Ca' Foscari Venezia.
https://www.dsi.unive.it/~gasparetto/materials/MIPS_Instruction_Set.pdf

MIPS Technologies. (2016). MIPS® architecture for programmers volume II-A: The MIPS32® instruction set manual (Rev. 6.06). MIPS Technologies, Inc.

